

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Ingegneria del Software: Tecniche Avanzate

Progetto ISTA – Evoluzione di CodeSmile Report Finale

Professore:

**Ch.mo Prof.
Andrea De Lucia**

Tutor di riferimento:

**Giammaria Giordano
Gilberto Recupito**

Studenti:

**Antonio D'Auria
Mat. NF22500063
Emanuele D'Auria
Mat. 0522501918
Luca Casillo
Mat. NF22500037**

ANNO ACCADEMICO 2025/2026

INDICE

Indice

Elenco delle tabelle

1	Introduzione	1
1.1	Scopo	1
1.2	Perimetro	1
1.3	Riferimenti Documentali	1
2	Evoluzione Realizzata	2
2.1	Coerenza tra le Modifiche	3
2.2	Compatibilità con la Baseline	3
3	Tracciabilità dell'Impatto	3
3.1	Accuratezza dell'Analisi	4
3.2	Gestione degli Scostamenti	4
3.3	Collegamento tra Impatto e Verifica	4
4	Validazione Consolidata	5
4.1	Esiti delle Verifiche	5
4.2	Non-Regression e Incidenti	6
4.3	Criteri di Accettazione	6
5	Chiusura del Progetto	7
5.1	Stato di Consegna	7
5.2	Aspetti Residui	7
5.3	Conclusioni	8

ELENCO DELLE TABELLE

1.1 Ruolo dei documenti nel processo di evoluzione	2
2.1 Sintesi delle Change Request implementate	3
3.1 Sintesi degli insiemi e delle metriche di impatto	4
3.2 Tracciabilità tra impatto e verifica	5
4.1 Esiti consolidati delle verifiche post-modifica	5
4.2 Indicatori conclusivi di non-regressione	6
4.3 Applicazione conclusiva dei criteri di accettazione	7

1 Introduzione

Il presente documento conclude il percorso di evoluzione di CodeSmile e ne consolida i risultati. La finalità non è riprodurre le analisi contenute negli elaborati precedenti, ma ricondurre baseline, Change Request, impatti e verifiche alla decisione conclusiva sullo stato del sistema.

Il report fornisce quindi una vista unitaria del lavoro svolto, esplicita la tracciabilità tra requisiti ed evidenze e registra gli aspetti residui che non pregiudicano l'accettazione. I dettagli architetturali, le matrici di impatto e i risultati completi delle singole suite restano affidati ai rispettivi documenti specialistici.

1.1 Scopo

Gli obiettivi del Report Finale sono:

- ricostruire in forma sintetica il passaggio dallo stato iniziale alla configurazione post-modifica;
- mostrare la chiusura delle quattro Change Request rispetto ai deliverable dichiarati;
- collegare gli impatti effettivi alle verifiche eseguite;
- applicare in modo conclusivo i criteri di accettazione stabiliti nel Master Test Plan;
- formulare una decisione motivata sulla chiusura del progetto nel perimetro concordato.

1.2 Perimetro

La valutazione comprende lo Static Analysis Tool, la CLI, il flusso statico dei servizi backend, la generazione dei report, la Web Application e le modalità di esecuzione locale e Docker interessate dalle modifiche. La semantica dell'AI-Based Detection Tool, i modelli utilizzati e la qualità delle relative predizioni restano fuori scope, in continuità con i documenti di testing e con il Master Test Plan. Tale componente non è destinatario delle Change Request e gli scenari che dipendono dal servizio AI esterno non concorrono alla decisione di accettazione.

1.3 Riferimenti Documentali

Gli elaborati prodotti formano una sequenza in cui ciascun documento assume una responsabilità distinta. La Tabella 1.1 ne sintetizza il contributo alla chiusura del progetto.

Tabella 1.1. Ruolo dei documenti nel processo di evoluzione

Doc.	Elaborato	Contributo
01	Reverse Engineering	Definisce la baseline architetturale e funzionale del sistema pre-modifica.
02	Pre-Modification System Testing	Delimita il perimetro, i livelli di test e la baseline dei casi di sistema.
03	Pre-Modification Test Incident Report	Registra gli incidenti iniziali, le correzioni e gli esiti non riconducibili a fault.
04	Change Requests	Specifica motivazioni, categorie e deliverable di CR1–CR4.
05	Master Test Plan	Stabilisce strategia, valori di coverage di riferimento e criteri di accettazione.
06	Impact Analysis	Confronta impatto atteso ed effettivo mediante SIS, CIS, AIS, FPIS e DIS.
07	Post-Modification Testing	Raccoglie le evidenze di conformità, regressione e accettazione post-modifica.

Il presente elaborato utilizza tali risultati come evidenze consolidate: ogni decisione riportata nelle sezioni successive è pertanto riconducibile a un requisito, a un insieme di impatto o a un’attività di verifica già documentata.

2 Evoluzione Realizzata

La baseline descritta nel Reverse Engineering offriva già un insieme di regole AST per rilevare sedici ML-specific code smell, modalità di esecuzione CLI e GUI e una Web Application articolata in gateway e servizi. Il sistema non produceva tuttavia una rappresentazione delle relazioni di chiamata, la CLI non esponeva opzioni dedicate a tale artefatto, il passaggio tra ambiente locale e Docker richiedeva adattamenti manuali e il frontend non disponeva di una vista strutturale dei risultati.

Le quattro Change Request hanno affrontato questi limiti mantenendo invariata la responsabilità delle detection rule e preservando i flussi preesistenti. La Tabella 2.1 riporta il risultato finale di ciascun intervento.

Tabella 2.1. Sintesi delle Change Request implementate

CR	Categoria	Obiettivo	Risultato finale	Stato
CR1	Enhancement	Generare il Call Graph mediante analisi dell'AST.	Nuovo package dedicato, artefatto JSON e integrazione con Inspector e ProjectAnalyzer per analisi singole e multiple.	Accettata
CR2	Perfettiva	Rendere configurabile dalla CLI la nuova capacità.	Opzioni <code>-enable-callgraph</code> , <code>-callgraph-output</code> , <code>-exclude-paths</code> e <code>-format</code> ; validazione, propagazione e documentazione aggiornate.	Accettata
CR3	Adattiva	Uniformare l'esecuzione locale e Docker.	Default locali, override tramite variabili d'ambiente, import e percorsi portabili, build context controllato e istruzioni di avvio aggiornate.	Accettata
CR4	Enhancement	Visualizzare il Call Graph nella Web Application.	Viewer interattivo con colorazione semantica, selezione, sidebar, filtri, controlli grafici ed esportazione JSON e PNG.	Accettata

2.1 Coerenza tra le Modifiche

Le Change Request costituiscono un percorso incrementale. CR1 introduce il dato strutturale nel core; CR2 lo rende attivabile e configurabile dalla CLI; CR4 ne estende il trasporto fino al frontend e lo presenta insieme agli smell testuali. CR3 agisce trasversalmente sulla configurazione dei servizi e rende lo stesso flusso eseguibile nei due ambienti previsti senza modificare il codice.

La separazione delle responsabilità evita che la visualizzazione dipenda da una seconda implementazione del grafo: il frontend consuma il contratto `graph_data` prodotto dal servizio statico, a sua volta basato sull'artefatto introdotto nel core. Gli attributi `smells`, `is_smelly` e `calls_smelly` collegano le informazioni di detection ai nodi senza alterare la forma dei report preesistenti.

2.2 Compatibilità con la Baseline

La generazione del Call Graph rimane opzionale e il formato CSV continua a essere il default della CLI. Le modalità di analisi precedenti conservano quindi il proprio comportamento quando i nuovi flag non sono specificati. Nella Web Application la rappresentazione grafica affianca, anziché sostituire, l'elenco degli smell; nel backend gli endpoint configurabili mantengono gli stessi contratti osservabili. Queste scelte hanno limitato la propagazione delle modifiche e reso possibile un confronto diretto con la baseline pre-modifica.

3 Tracciabilità dell’Impatto

L’Impact Analysis ha verificato il confine delle modifiche prima di indirizzare le attività di regressione. Il confronto tra Candidate Impact Set e Actual Impact Set ha permesso di

distinguere le dipendenze considerate in via cautelativa dalle unità emerse soltanto durante l’implementazione.

3.1 Accuratezza dell’Analisi

La Tabella 3.1 riprende esclusivamente gli indicatori necessari alla valutazione conclusiva. Le cardinalità sono riferite alle singole Change Request; la stessa unità può comparire in più interventi quando assume responsabilità differenti.

Tabella 3.1. Sintesi degli insiemi e delle metriche di impatto

CR	SIS	CIS	AIS	FPIS	DIS	Precision	Recall
CR1	5	8	5	3	0	0,63	1,00
CR2	4	6	4	2	0	0,67	1,00
CR3	5	11	9	3	1	0,73	0,89
CR4	11	15	13	3	1	0,80	0,92

Nel complesso, 29 occorrenze di impatto effettivo erano comprese nelle 40 candidature, a fronte di 31 occorrenze effettive. La Precision aggregata è quindi pari a 0,73 e la Recall a 0,94. Il modello ha privilegiato una copertura elevata degli impatti reali mantenendo circoscritto il numero di falsi positivi. La Recall completa di CR1 e CR2 conferma inoltre che l’estensione del core e della CLI è rimasta entro il confine previsto.

3.2 Gestione degli Scostamenti

I due Discovered Impact Set riguardano livelli diversi. Per CR3 è emersa la necessità del file `.dockerignore`, utilizzato per controllare il build context; per CR4 è stato coinvolto `FileInput`, affinché la selezione multipla conservasse i percorsi relativi richiesti dal grafo. Entrambe le unità sono state incluse nelle verifiche post-modifica: la prima nel build e negli smoke test dei container, la seconda nei test Jest e Cypress.

Le unità appartenenti ai False Positive Impact Set non hanno richiesto interventi, ma sono rimaste nel perimetro di regressione. In questo modo un falso positivo dell’analisi non è stato interpretato come irrilevante: CLI, GUI, FileUtils, Inspector, reporting, Dockerfile, package Call Graph e contesto del frontend sono stati esercitati nei livelli di test coerenti con le rispettive responsabilità.

3.3 Collegamento tra Impatto e Verifica

La Tabella 3.2 mostra come gli elementi determinanti di ciascun AIS e gli scostamenti dell’analisi siano confluiti nelle evidenze di chiusura. Il dettaglio dei file rimane nell’Impact Analysis; il dettaglio dei casi e degli esiti nel Post-Modification Testing.

Tabella 3.2. Tracciabilità tra impatto e verifica

CR	Impatto determinante	Evidenza conclusiva	Esito
CR1	Package Call Graph, Inspector e ProjectAnalyzer.	Test unitari di extractor e builder; integrazione delle pipeline singola e multipla; regressione dei consumer.	PASS
CR2	CLI, orchestratore, FileUtils e README.	Test di parsing, validazione e propagazione; controllo dei formati, dei percorsi esclusi e della documentazione.	PASS
CR3	Configurazione, gateway, servizi statico e report, build context.	Test locali, override d'ambiente, validazione Compose, build delle immagini e smoke HTTP.	PASS
CR4	Contratto full-stack, viewer, tipi condivisi, Inspector e FileInput.	Test Jest dei componenti, integrazione del servizio statico e flusso Cypress con interazioni ed esportazioni.	PASS

4 Validazione Consolidata

La validazione post-modifica ha seguito la progressione definita nel Master Test Plan, dalle unità fino ai flussi end-to-end e alle verifiche infrastrutturali. La Tabella 4.1 riporta gli esiti necessari alla decisione finale senza ripetere i casi di test e le osservazioni operative già documentati.

4.1 Esiti delle Verifiche

Tabella 4.1. Esiti consolidati delle verifiche post-modifica

Suite o controllo	Risultato	Valutazione	Esito
Unit test Python	175 / 175	Core statico, CLI, Call Graph, GUI e report; branch coverage 87%.	PASS
Integration test Python	9 / 9	Pipeline del core, CLI, GUI e contratto Call Graph; branch coverage 51%.	PASS
System test CLI	21 casi	14 PASS, 3 PASS_NO_OUTPUT e 4 SKIP_NO_PY; classificazioni e output invariati rispetto alla baseline.	PASS
Web integration	5 / 5 in scope	Contratti di analisi statica, reporting e configurazione; 2 scenari AI classificati N/A.	PASS
Frontend Jest	32 / 32	Sette suite, incluse le verifiche del viewer e della pagina di upload.	PASS
Frontend build	1 / 1	Controlli TypeScript e lint; generazione statica di nove pagine.	PASS
Cypress End-to-End	22 / 22 in scope	Ventiquattro scenari registrati; 2 scenari AI pending e classificati N/A.	PASS
Docker e smoke HTTP	Configurazione valida	Tre immagini e tre servizi in scope; flussi statico e report con risposta HTTP 200.	PASS

Non sono stati osservati fallimenti riproducibili nelle componenti in scope. Le esclusioni riguardano esclusivamente il servizio AI esterno e non derivano da test falliti sulle funzionalità oggetto delle Change Request.

4.2 Non-Regression e Incidenti

Il confronto con la baseline mostra che la crescita funzionale non è stata ottenuta riducendo le garanzie esistenti. Gli indicatori principali sono riportati nella Tabella 4.2.

Tabella 4.2. Indicatori conclusivi di non-regressione

Indicatore	Baseline	Stato finale	Valutazione
Branch coverage unit	85%	87%	Preservata e incrementata.
Branch coverage integration	34%	51%	Preservata e incrementata.
System test CLI	14 PASS, 3 PASS_NO_OUTPUT, 4 SKIP_NO_PY, 0 FAIL	Stessa distribuzione e stessi risultati	Nessuna regressione puntuale.
Incidenti in scope	6 incidenti risolti	6 incidenti ancora chiusi; nessuna riapertura	Nessun nuovo incidente registrato.

Gli incidenti confermati come chiusi sono INC-SAU-01, INC-SAU-02, INC-WAU-01–INC-WAU-03 e INC-WAE-01. INC-WAU-04, INC-WAU-05 e INC-WAE-02 rimangono N/A perché dipendenti dalla modalità AI-Based fuori scope. Questa classificazione conserva gli stessi identificativi e la stessa separazione per componente e livello introdotte nel report pre-modifica.

La test suite è stata estesa senza eliminare file: rispetto ai 46 file della baseline ne risultano 53, con 7 file aggiunti, 16 modificati e 30 invariati. L’assenza di eliminazioni e la riesecuzione delle suite preesistenti forniscono ulteriore evidenza della continuità della strategia di regressione.

4.3 Criteri di Accettazione

La decisione finale applica i quattro criteri definiti nel Master Test Plan. Le evidenze della Tabella 4.3 rinviano agli indicatori già consolidati, evitando di introdurre soglie o condizioni ulteriori.

Tabella 4.3. Applicazione conclusiva dei criteri di accettazione

Criterio	Evidenza consolidata	Esito
Preservazione della coverage	Entrambi i valori misurabili superano le rispettive baseline.	Soddisfatto
Assenza di bug critici	Tutte le verifiche in scope terminano senza fallimenti e nessun incidente risolto è stato riaperto.	Soddisfatto
Allineamento ai contratti esistenti	CLI, gateway, servizio statico e servizio di report conservano i comportamenti osservabili attesi in locale e Docker.	Soddisfatto
Assenza di regressioni funzionali	Corpus di sistema invariato, suite frontend positive e flussi end-to-end in scope completati.	Soddisfatto

Tutti i criteri risultano soddisfatti. L'accettazione comprende i deliverable documentali di CR2 e CR3, la verifica dei DIS di CR3 e CR4 e la regressione dei componenti presenti nei FPIS.

5 Chiusura del Progetto

5.1 Stato di Consegna

Lo stato finale di CodeSmile soddisfa il perimetro di evoluzione definito nel documento di Change Requests. Il core statico produce un Call Graph strutturato e correlato agli smell; la CLI ne controlla generazione, destinazione, percorsi esclusi e formato del report; i servizi in scope sono configurabili senza modifiche al codice in ambiente locale e Docker; la Web Application presenta il grafo con le interazioni, i filtri e le esportazioni richieste.

CR1, CR2, CR3 e CR4 risultano implementate, verificate e accettate. I deliverable funzionali, adattivi e documentali sono presenti e le evidenze raccolte soddisfano i criteri decisionali stabiliti prima della modifica. La baseline rimane riproducibile e i nuovi comportamenti sono coperti a livello unitario, di integrazione, di sistema o end-to-end secondo la componente interessata.

5.2 Aspetti Residui

Non risultano difetti critici o bloccanti nel perimetro del progetto. Gli avvisi osservati durante il testing riguardano API o configurazioni deprecate di HTTPX, Node.js, Docker Compose e Pydantic, oltre a un avviso di risoluzione della configurazione Cypress. Essi non alterano gli esiti funzionali e costituiscono debito tecnico di manutenzione, non condizioni che impediscono la chiusura.

Il componente AI-Based mantiene lo stesso stato di esclusione adottato fin dalla pianificazione. Una sua futura inclusione richiederebbe un ambiente riproducibile per il servizio esterno, oracoli dedicati e criteri di accettazione separati; tale attività rappresenterebbe un nuovo perimetro e non un'estensione della validazione corrente.

Come possibili attività successive, senza impatto sull'accettazione raggiunta, rimangono:

- aggiornare progressivamente dipendenze e configurazioni interessate dagli avvisi di deprecazione;
- automatizzare in una pipeline di integrazione continua i quality gate già applicati localmente, inclusi test, coverage, build e smoke test;
- definire un piano autonomo per il servizio AI soltanto qualora esso venga incluso in una futura evoluzione del sistema.

5.3 Conclusioni

Il progetto ha seguito un processo completo di comprensione, definizione delle modifiche, pianificazione, analisi dell'impatto e verifica post-modifica. La tracciabilità tra i sette elaborati precedenti consente di motivare ogni scelta con una baseline, un requisito, un insieme di impatto o un'evidenza di test.

L'evoluzione introduce nuove capacità strutturali e di presentazione senza compromettere i comportamenti esistenti: la coverage è preservata, il corpus di sistema resta stabile, gli incidenti in scope rimangono chiusi e tutti i flussi funzionali rilevanti terminano positivamente. Su queste basi il progetto CodeSmile può essere considerato concluso con esito positivo nel perimetro delle quattro Change Request.